Shopify-Anbindung an Skipily ueber Make.com

Praezise Anleitung fuer den Produkt- und Bestell-Sync (Shop: marinedustrie.de, ein Shopify-Store). Stand: einfache Auth nur per x-api-key, lastfest mit Rate-Limit.

1. Architektur

Zwei Datenfluesse ueber Make.com:
Flow A - Produkte: Shopify -> Make -> Skipily Products-API (Produkte und Bestaende in den Skipily-Shop).
Flow B - Bestellungen: Skipily Order-Webhook -> Make -> Shopify (Bestellung/Notiz anlegen).

2. Voraussetzungen

1) **Skipily-Provider-API-Key** (Provider-Portal, Pro-Tarif). 2) **Endpoint:** POST/PUT https://vcjwlyqkfkszumdrfvtm.supabase.co/functions/v1/products-api. 3) Auth ausschliesslich per Header **x-api-key** (kein weiterer Schluessel noetig). 4) **Rate-Limit:** 300 Schreib-Anfragen/min je Provider, 600 Lese-Anfragen/min je IP; bei Ueberlast antwortet die API mit HTTP 429.

3. Flow A: Shopify -> Skipily

1) In Make ein Szenario anlegen und den Blueprint importieren (Datei make-blueprint-shopify-to-skipily.json). Der Trigger ist ein Custom-Webhook; dessen URL kopieren.
2) In Shopify unter Einstellungen -> Benachrichtigungen -> Webhooks zwei Webhooks (Format JSON) auf die Make-URL: 'Produkterstellung' (products/create) und 'Produktaktualisierung' (products/update).
3) Im HTTP-Modul den Platzhalter x-api-key durch deinen Skipily-Provider-Key ersetzen.

4. Feld-Zuordnung (Shopify -> Skipily)

Shopify	Skipily
title	name
vendor	manufacturer
body_html (HTML entfernt)	description
variants[].price	price
variants[].sku	sku / part_number
variants[].barcode	ean
variants[].inventory_quantity	stock_quantity / in_stock
variants[].grams / 1000	weight_kg
images[].src	images[]
product_type / tags	category_id (Mapping) / tags
status = active	is_active

5. Request-Body (einfaches Produkt)

```
POST https://vcjwlyqkfkszumdrfvtm.supabase.co/functions/v1/products-api
Header:  Content-Type: application/json
         x-api-key: <DEIN_SKIPILY_PROVIDER_KEY>

{
  "sku": "{{1.variants[1].sku}}",
  "name": "{{1.title}}",
  "manufacturer": "{{1.vendor}}",
  "description": "{{ HTML entfernt aus 1.body_html }}",
  "price": {{1.variants[1].price}},
  "currency": "EUR",
  "ean": "{{1.variants[1].barcode}}",
```

```
  "stock_quantity": {{1.variants[1].inventory_quantity}},
  "in_stock": {{ inventory_quantity > 0 }},
  "weight_kg": {{1.variants[1].grams / 1000}},
  "images": ["{{1.image.src}}"],
  "category_id": null,
  "is_active": true
}
```

6. Request-Body mit Varianten

variant_options = Shopify-Optionen (Name + Werte). variants = eine Zeile je Shopify-Variante. variants ersetzt beim POST/PUT ALLE vorhandenen Varianten.

```
{
  "name": "Liros Race SK78", "manufacturer": "Liros",
  "currency": "EUR", "price": 100.0, "sku": "LIROS-SK78",
  "variant_options": [{"name":"Durchmesser","values":["6mm","8mm","10mm"]}],
  "variants": [
    {"attributes":{"Durchmesser":"6mm"},"price":100.0,"stock_quantity":20,"sku":"SK78-6"},
    {"attributes":{"Durchmesser":"8mm"},"price":120.0,"stock_quantity":15,"sku":"SK78-8"}
  ]
}
```

7. Flow B: Skipily -> Shopify (Bestellungen)

1) In Make einen zweiten Custom-Webhook anlegen. 2) Dessen URL im Provider-Profil als webhook_url hinterlegen. 3) Skipily POSTet bei Bestell-Ereignissen signiert dorthin (siehe unten). In Make daraus eine Shopify-Bestellung (Admin-API) oder Notiz/Mail erzeugen.

```
POST an service_providers.webhook_url (im Provider-Profil)
Header:  X-BoatCare-Event, X-BoatCare-Signature (HMAC)

{
  "event_type": "order_created",
  "order": {
    "order_number": "SK-2026-000123", "total": 129.9, "currency": "EUR",
    "buyer_name": "Max Mustermann", "shipping_city": "Bonn",
    "line_items": [
      {"product_name":"Liros Race SK78","product_sku":"LIROS-SK78",
       "variant_sku":"SK78-8","variant":{"Durchmesser":"8mm"},
       "quantity":2,"unit_price":120.0,"total":240.0}
    ]
  }
}
```

8. Hinweise

Kategorien: Skipily nutzt category_id (UUID). Liste per GET .../rest/v1/product_categories?select=id,name_de,slug (Header apikey:). Ohne Mapping bleibt category_id null (Produkt unkategorisiert, funktioniert).
Bestaende: products.json enthaelt keine Lagermenge; der products/update-Webhook und die Shopify-Admin-API liefern inventory_quantity. Optional zusaetzlich Webhook inventory_levels/update.
Neu vs. Update: POST legt an, PUT ?id= aktualisiert. Fuer sauberes Update die Skipily-uuid je Shopify-Produkt in einem Make Data Store merken.